

## Programación Concurrente

### Actividad Práctica (Opcional)

#### Procesos Livianos

##### C++

##### **El entrenamiento de Naruto Uzumaki**



Naruto Uzumaki dispone de poco tiempo para subir de nivel su técnica *Rasengan* antes de que llegue la siguiente amenaza a la aldea. Para acelerar el proceso de entrenamiento utilizará su Jutsu *Multi Clones de Sombra*, permitiendo que cada clon realice una parte del entrenamiento y al final Naruto incorpore el avance de cada clon como propio.

##### Sobre los clones se sabe lo siguiente:

- La cantidad de Chakra (energía) para entrenar es variable (5 a 10 unidades).
- Al consumir todo el Chakra el clon desaparece finalizando su sesión de entrenamiento.
- La sesión de entrenamiento de cada clon se compone de varios intentos (uno por cada unidad de Chakra).
- La duración de un intento es variable (100 a 200 milisegundos).
- Cada intento puede salir bien (se sube un punto de nivel) o salir mal (no se sube de nivel). Hay una probabilidad del 0.5 de subir de nivel.

##### Se pide:

- Implementar un programa que simule el entrenamiento de Naruto utilizando

programación concurrente a través de N hilos, permitiendo tomar el tiempo de ejecución.

- No utilizar ningún mecanismo de comunicación ni sincronización entre los hilos.
- Ejecutar el programa para distintas cantidades de clones (5, 10, 20, etc). Informando el tiempo y nivel alcanzado para cada ejecución.
- Responda: ¿Cuál sería la cantidad óptima de clones? Justifique su respuesta.
- Realizar un gráfico de tiempo en base a la cantidad de clones.

##### Python

##### **Combates Eternos en el Netherrealm**



Se ha organizado una nueva edición del torneo de Mortal Kombat II, en el cual participan distintos guerreros que se enfrentarán para determinar quién es el mas fuerte.

Debido a la gran cantidad de combates que deben analizarse, se desea desarrollar un programa que permita simular concurrentemente una gran cantidad de torneos independientes, con el objetivo de reducir el tiempo total de procesamiento.

Cada simulación de un torneo es completamente independiente de las demás, por lo que puede ser ejecutada concurrentemente sin necesidad de que los hilos intercambien información durante su ejecución.

La modalidad del torneo es eliminatorio (Cuartos de final – Semifinal – Final). Para cada torneo se seleccionarán aleatoriamente 8 luchadores diferentes de los 12 disponibles. Todos los luchadores deberán tener la misma probabilidad de ser seleccionados. La lista posible de luchadores es:

|          |             |             |         |
|----------|-------------|-------------|---------|
| Liu Kang | Kung Lao    | Johnny Cage | Reptile |
| Sub-Zero | Shang Tsung | Kitana      | Jax     |
| Mileena  | Baraka      | Scorpion    | Raiden  |

Cada guerrero posee determinados atributos (queda a criterio del grupo definir el valor de cada atributo, siempre y que sea dentro del rango especificado):

| Atributo         | Función                                              | Rango       |
|------------------|------------------------------------------------------|-------------|
| <b>Vida</b>      | Cantidad de daño máximo que puede recibir            | 80 a 120    |
| <b>Ataque</b>    | Daño base                                            | 15 a 30     |
| <b>Defensa</b>   | Reduce el daño recibido                              | 5 a 15      |
| <b>Velocidad</b> | Determina quien ataca primero                        | 1 a 10      |
| <b>Crítico</b>   | Probabilidad de daño x 2                             | 0,05 a 0,25 |
| <b>Bloqueo</b>   | Probabilidad de bloquear el ataque y no recibir daño | 0,05 a 0,25 |

El daño base se calcula como:

$$\text{Daño} = \text{ataque} - \text{defensa}$$

El daño mínimo producido por un ataque exitoso será de 1 punto. Antes de aplicar el daño, se determinará aleatoriamente si el

defensor logra bloquear el ataque. En caso de bloqueo, el daño será 0.

Si el ataque no es bloqueado, se determinará aleatoriamente si se produce un golpe crítico. En caso de crítico, el daño calculado será multiplicado por 2.

Al comenzar cada combate, el luchador con mayor velocidad será quien realice el primer ataque. En caso de empate, se determinará aleatoriamente quién comienza. Luego los participantes alternarán sus turnos hasta que uno de ellos quede sin puntos de vida.

Para cada torneo simulado, los participantes se seleccionan aleatoriamente y se enfrentan en una serie de combates.

Un combate se desarrolla mediante turnos. En cada turno, uno de los guerreros realiza un ataque sobre su oponente. El combate finaliza cuando uno de los participantes pierde todos sus puntos de vida.

El ganador continúa participando del torneo hasta que finalmente se determina un campeón.

### Concurrencia

Para acelerar la ejecución se deberán utilizar N hilos. Cada hilo será responsable de realizar una determinada cantidad de simulaciones completas del torneo. Por ejemplo, si deben realizarse 100.000 simulaciones y se utilizan 4 hilos, cada hilo realizará 25.000 simulaciones. El programa debe permitir definir la cantidad total de torneos a simular como así también la cantidad de hilos.

### Durante la ejecución:

Los hilos no podrán comunicarse ni sincronizarse entre sí. Cada hilo deberá trabajar con sus propios datos. El único mecanismo permitido para coordinar la finalización será join() desde el hilo principal.

### Resultados

Cada hilo deberá almacenar sus resultados en una estructura independiente.

Como mínimo, deberá registrar:

- Cantidad de turnos simulados.
- Cantidad de victorias de cada guerrero.
- Cantidad de veces que cada guerrero resultó campeón.

### **Medición del tiempo**

El programa deberá medir el tiempo necesario para completar todas las simulaciones. Se deberá experimentar con distintas cantidades de hilos, manteniendo fija la cantidad de simulaciones.

Se recomienda realizar varias ejecuciones para cada cantidad de hilos y calcular el tiempo promedio, debido a la variabilidad propia de la ejecución concurrente.

### **Análisis**

A partir de los resultados obtenidos se deberá:

1. Realizar un gráfico de cantidad de hilos vs. tiempo de ejecución.
2. Determinar cuál fue la cantidad de hilos que obtuvo el menor tiempo.
3. Analizar si aumentar indefinidamente la cantidad de hilos produce una mejora en el rendimiento.